

NEXUS Native Compiler

The NEXUS toolchain was delivered without its compiler. This report documents the ground-up rebuild of nexc as a single-pass AOT compiler emitting standalone x86-64 machine code for both Linux ELF64 and Windows PE32+, and the full verification results: 12 example suites, 13 diagnostics, 38 foundation tests, 3 interactive tests, and 94 standard-library assertions.

Version v5.8

Scope Compiler repair, print_fmt, regression suite

Date September 22, 2026

Contents

Contents	1
1. Executive Summary	1
2. Diagnosis of the Delivered Archive	2
3. Rebuild Approach and Architecture	3
4. Verification Results	4
5. Improvements Beyond Repair	5
6. Known Limitations and Recommended Next Steps	6

1. Executive Summary

The NEXUS project archive delivered for improvement contained the documentation, standard library, example programs, and Linux tool binaries of an ambitious self-hosting compiler project - but not the compiler itself. Both the compiler binary (`nexc.elf`) and its source (`nexc.nex`) were absent, together with the C sources of the auxiliary tools, the test suite, and several examples referenced by the driver script. As a result, every compile path failed on arrival: `./nexus run` aborted because the compiler working directory did not exist, and no example could be executed.

This report documents the repair: a complete compiler was rebuilt from scratch in C (approximately 2,600 lines), targeting the same NEXUS language surface documented in the project handbook and exercised by the preprocessor and tool binaries that did survive. The rebuilt compiler emits standalone native x86-64 machine code for two operating systems from one invocation - a Linux ELF64 binary driven by raw kernel syscalls, and a Windows PE32+ binary importing exactly seven kernel32 functions - with no runtime, no virtual machine, and no libc. The rebuild preserves the project's founding invariant of 0 percent C# and 0 percent .NET runtime in every emitted binary.

Verification is comprehensive and passing. The driver's embedded regression suite reports 12 of 12 runnable example suites, 13 of 13 compiler diagnostics, 38 of 38 foundation tests (including two new `print_fmt` interpolation tests), 3 of 3 interactive input tests, and a 94-assertion standard-library suite covering all 41 library functions. Compilation is deterministic: two runs on identical input produce byte-identical binaries, verified by SHA-256 digest comparison in the `./nexus self-host` command. One new language feature, compile-time string interpolation via `print_fmt`, was implemented and is demonstrated with a dedicated example.

2. Diagnosis of the Delivered Archive

The archive contained 45 files in five directories. Triage against the references in the driver script, the roadmap, and the architecture handbook produced the following inventory of present, working, and missing assets. The single decisive finding was the absence of the compiler: every remaining tool either consumes compiler output or prepares compiler input, so the pipeline was dead on arrival.

Component	Status	Evidence
bin/ tools (nexload, nexprep, nexelf, nextest, nexfmt)	Present, runnable	ELF64 binaries execute on this host
stdlib/ - 11 source modules	Present	nstdlib umbrella includes 5 modules, 41 functions
examples/ - 10 NEXUS programs	Present (4 referenced ones missing)	fibonacci, finance, raycaster_3d, ...
nexus driver script	Present, 589 lines	run/compile/test/repl/new/fmt/self-host commands
compiler/ (nexc.elf binary + nexc.nex source)	Missing	driver fails: cannot write compiler/code.nex
tools/ C sources	Missing	driver rebuild logic references nexload.c, nexprep.c, ...
tests/stdlib_test.nex (94 assertions)	Missing	harness references tests/stdlib_test.nex
examples/modules/, prime_checker.nex	Missing	foundation tests include math_utils.nex
docs/, editors/, CHANGES.md	Missing	referenced by roadmap

Table 1: Archive triage. Bold rows block the toolchain or the test suite.

Two failure modes were confirmed by execution. First, the driver's run and test commands terminated with an inability to write compiler/code.nex, because the compiler working directory itself was absent. Second, even with that directory created by hand, no compiler binary existed to consume the preprocessed source. The recovery plan therefore centered on supplying a compiler that accepts the exact language surface the surviving preprocessor produces, together with restoration of the missing test and example files.

3. Rebuild Approach and Architecture

The rebuilt compiler, `tools/nexc.c`, is a single-pass ahead-of-time compiler written in C and built with `gcc -O2`, matching the convention already used by the surviving tools in `bin/`. It reads the preprocessed translation unit (`code.nex`, produced by `nexprep`), parses it once, emits x86-64 machine code directly into an in-memory buffer, and writes two complete executables. No intermediate assembly, no external assembler, and no linker participate: header synthesis for both PE32+ and ELF64 is performed inside the compiler itself.

Element	Design
Variable model	Single global frame; every variable is one 8-byte slot addressed as <code>[rbp - 8*(idx+1)]</code> ; 1,024 slots
Subroutines	Legacy <code>fn/call</code> sharing the global frame; parameterized calls flattened by <code>nexprep</code> into <code>_arg_*/_ret_*</code> slot copies
Expression evaluation	Strict left-to-right fold into RAX with operand in RBX; mid-chain atoms spilled through a temporary slot
Floats	SSE2 doubles (<code>addsd/subsd/mulsd/divsd/sqrtsd</code> , <code>ucomisd</code> branches for <code>if_f/while_f</code>)
Conditions	operand OP operand with inverted-branch backpatching (<code>jne/je/jg/jle/jge/jle</code>); strict grammar rejects chained arithmetic as documented
Fixups	User code emitted once with symbolic fixups (helper calls, function calls, RIP-relative strings and constants), resolved per target layout
Runtime helpers	17 embedded routines per target: <code>print_int</code> , <code>print_str</code> , <code>print_seg</code> , <code>print_nl</code> , <code>fmt_int</code> , <code>fmt_float</code> , <code>print_float</code> , <code>read_int</code> , <code>alloc</code> , <code>exit</code> , <code>file_create/open/read/write/close</code> , <code>len</code> , <code>itoa</code>
Diagnostics	Line/column reporting with the exact message strings the regression suite matches; no output files on any error

Table 2: Compiler architecture decisions and their rationale.

The two binary formats differ only in layout and system interface. The ELF64 image is a single `RWX PT_LOAD` segment at `0x400000` with the entry point immediately after the 120-byte header pair, and the runtime performs work through raw syscalls. The PE32+ image places prologue, user code, and runtime in `.text` with a `.rdata` section holding the constant pool, import descriptors, and string pool, and the runtime calls seven `kernel32` functions through the Import Address Table. Because fixups are resolved per target, both images share byte-identical user machine code. Emitted programs now terminate lines with LF on both platforms; the historic CRLF emission was dropped, which removes the need for the test harness's `tr -d '\r'` workaround.

```

mov rbp, rsp ; prologue: frame + zeroed slots
rep stosq
call print_int ; E8 rel32 into the embedded runtime
mov rax, 60 ; sys_exit
syscall ; bare-metal termination (Linux)

```

Illustrative code patterns emitted by the rebuilt compiler.

4. Verification Results

Verification ran the driver's embedded regression suite end to end after the compiler was installed and the harness was updated to invoke it directly. The suite exercises the toolchain through the same pipeline users will: nexprep preprocessing, native compilation, and direct execution of the emitted ELF64 on the Linux kernel. All gates pass, and the two failures observed mid-rebuild were traced to defects in the rebuilt compiler and fixed before final runs.

Gate	Result	Notes
Example suites	12 / 12 PASS	fibonacci, finance, rocket_physics, string_demo, functions_demo, stress_100_vars, stdlib_demo, struct_demo, builtins_demo, multi_file_demo, print_fmt_demo, full_demo
Compiler diagnostics	13 / 13 PASS	Unrecognized statement, missing '=', undefined call, unclosed/unmatched block, condition traps, break/continue outside loop, for header, read operand - each gated with no output files
Foundation regression	38 / 38 PASS	Negative literals and comparisons, else-if chains, strings, len/abs, includes, structs, for/break/continue, store64 scale, compound assignment, array literals, function parameters, floats, print_fmt
Interactive input	3 / 3 PASS	interactive_calc (45+15), prime_checker 17 and 24, stream-preserving read
Standard library suite	94 / 94 assertions	All 41 stdlib functions: math (24), arrays (16), strings (15), qarrays (16), built-ins and floats (23)
Determinism (self-host)	PASS	Two compilations of identical input yield byte-identical app.elf and app.exe (SHA-256 verified)
PE32+ structure	PASS	objdump-validated headers: entry RVA 0x1000, ImageBase 0x140000000, three sections, well-formed kernel32 import chain

Table 3: Full regression results of the rebuilt toolchain.

The raycaster example deserves a specific note. It compiles cleanly to both ELF64 and PE32+ and its output demonstrates correct fixed-point math and raw X11 protocol handling; at run time it intentionally exits with a diagnostic when no X11 server socket is available, which is the expected behavior on a headless machine. It was therefore excluded from the automated suite's execution gate and verified by compilation instead. The interactive tests confirm that the stream-preserving read implementation consumes exactly one line per read, so piped input such as 45 and 15 produces the expected sum, difference, product, quotient, and remainder.

5. Improvements Beyond Repair

Beyond restoring the toolchain to its documented state, the rebuild delivered targeted improvements requested for this pass and several quality fixes that fell out of the verification work. Each item below is covered by the regression suite or by a dedicated example.

Improvement	Detail
print_fmt string interpolation	Compile-time formatting: <code>print_fmt "fib({n}) = {result}!"</code> inserts integer, float (6-decimal, SSE2 dtoa), and string placeholders; <code>{{</code> and <code>}}</code> escape literal braces; demonstrated in <code>examples/print_fmt_demo.nex</code> and two foundation tests
Deterministic builds	Identical input yields byte-identical output; <code>./nexus self-host</code> now proves this via SHA-256 comparison, replacing the self-hosting fixed-point check that required the lost <code>nexc.nex</code>
LF line endings	Emitted programs print LF on both targets, removing the CRLF artifacts that previously required <code>tr -d '\r'</code> in harnesses
Compiler from source	<code>tools/nexc.c</code> ships with the project; the driver rebuilds <code>compiler/nexc.elf</code> automatically when missing or older than the source
Larger variable capacity	1,024 stack slots (from 255), which the 94-assertion <code>stdlib</code> suite with per-assertion temporaries exercises
assert.nex summary fix	<code>nx_assert_summary</code> printed duplicated fragments due to buffer aliasing in <code>nx_str_concat</code> ; rewritten with alternating destination buffers
Scaffolded projects build	<code>nexus new</code> now bakes the toolchain path into the generated <code>build.sh</code> ; the scaffold compiles and runs out of the box
REPL restored	<code>tools/nexrepl.sh</code> implements the <code>:run</code> / <code>:clear</code> / <code>:list</code> / <code>:quit</code> session commands the driver's repl entry expected
Harness hardening	Gated compilations no longer crash the test harness; <code>raycaster_3d</code> documented and skipped (requires an X11 server)

Table 4: Improvements delivered alongside the toolchain repair.

6. Known Limitations and Recommended Next Steps

Three limitations are inherited from the architecture rather than introduced by the rebuild, and each is documented in the README. Raw syscallN built-ins are Linux-only by construction and are rejected at run time on Windows; the X11 and framebuffer modules that use them are consequently bare-metal Linux modules. Parameterized functions flattened by nexprep share the single global frame, so legacy subroutines should not be invoked from inside a parameterized callee - the original design had the same constraint. Finally, one pre-existing quirk in the shipped nexprep binary removes bracketed sequences inside string literals; the affected demo labels were reworded, and the quirk is flagged in the verification log for a future nexprep source recovery.

Recommended next steps, in priority order: first, port the stdlib's x11 and sound modules to compile-time feature detection so PE32+ builds of GUI programs fail at compile time with a clear diagnostic rather than silently at run time. Second, recover or rewrite the VS Code grammar and docs/ directory referenced by the roadmap, which remain the largest documentation gaps. Third, extend the foundation suite with negative-path float coverage (division by zero, sqrt of negatives), which the current helper set maps to hardware semantics but does not yet report. Fourth, consider a Windows-hosted build path for the compiler itself so PE32+ output can be produced without a Linux toolchain present.